



AVALIAÇÃO 1

Unidade Curricular: Programação para Internet 1

CHAVE DE RESPOSTAS / GABARITO

Estudante: _____

Data: ____/____/____

Nota: _____

Instruções

- A prova contém 10 questões, divididas em Múltipla Escolha, V/F, Relacionar Colunas e Discursiva.
- Nas questões de Verdadeiro ou Falso, **justifique as alternativas falsas**.
- Leia atentamente cada enunciado antes de responder.

Parte 1: Múltipla Escolha

Questão 1: HTML5 Semântico

Qual tag semântica do HTML5 deve ser utilizada para englobar o conteúdo principal e exclusivo de uma página, excluindo cabeçalhos, rodapés e menus de navegação globais?

- a) <section>
- b) **<main>**
- c) <article>
- d) <header>



Questão 2: Layouts com CSS (Grid e Flexbox)

Sobre a diferença fundamental entre CSS Grid e Flexbox, assinale a alternativa **CORRETA**:

- a) **O CSS Grid foi projetado para layouts bidimensionais (linhas e colunas), enquanto o Flexbox foi projetado primariamente para layouts unidimensionais (uma linha ou coluna por vez).**
- b) Flexbox requer que o elemento pai tenha altura fixa, enquanto o Grid ajusta-se automaticamente.
- c) CSS Grid substituiu completamente o Flexbox, tornando-o obsoleto em navegadores modernos.
- d) O Flexbox utiliza `grid-template-columns` para alinhar elementos verticalmente.

Questão 3: Eventos no DOM

No JavaScript, qual método é a forma mais recomendada e moderna para adicionar um "escutador" de eventos a um botão selecionado, para executar uma função ao ser clicado?

- a) `button.onclick = function() { ... }`
- b) `button.addEventListener("click", function() { ... })`
- c) **`button.addEventListener("click", function() { ... })`**
- d) `<button onclick="function()">` diretamente no HTML

Questão 4: Assincronicidade (Fetch)

Qual é o principal retorno da função `fetch()` no JavaScript antes de receber os dados efetivos da requisição?

- a) Uma string no formato JSON.
- b) Uma matriz contendo os objetos da API.
- c) **Uma *Promise* (promessa) que representará a resposta HTTP no futuro.**
- d) Um erro temporário de bloqueio (*blocking error*).



Parte 2: Verdadeiro ou Falso (Justifique as falsas)

Questão 5: Box Model

(F) A propriedade `margin` do CSS adiciona espaço interno a um elemento, afastando o conteúdo das bordas da própria caixa. **Justificativa:** Falso. O `margin` adiciona espaço EXTERNO (entre a borda e outros elementos). O espaço INTERNO é o `padding`.

Questão 6: Escopo JS

(V) Variáveis declaradas com `let` e `const` possuem escopo de bloco, significando que não podem ser acessadas fora do bloco (`{ ... }`) onde foram definidas. **Justificativa:** Verdadeiro.

Questão 7: Responsividade

(F) A técnica *Mobile First* sugere que devemos escrever o CSS primeiramente para telas grandes (desktop) e depois usar Media Queries (`@media`) com `max-width` para ajustar o layout para celulares. **Justificativa:** Falso. A técnica Mobile First escreve o CSS base para celulares primeiro e usa Media Queries com `min-width` para adaptar o layout em telas maiores.

Questão 8: Manipulação DOM

(F) O método `document.querySelectorAll()` retorna o primeiro elemento HTML que corresponda ao seletor CSS fornecido. **Justificativa:** Falso. Ele retorna uma lista (`NodeList`) com TODOS os elementos que correspondem ao seletor. O método que retorna apenas o primeiro é o `querySelector()`.



Parte 3: Relacione as Colunas

Questão 9: Métodos DOM e API

Associe a Coluna A (Método) com a Coluna B (Descrição correta da sua função).

Coluna A:

- (1) `document.createElement('div')`
- (2) `elemento.appendChild(novoElemento)`
- (3) `response.json()`
- (4) `elemento.classList.add('ativo')`

Coluna B:

- (3) Transforma o fluxo de resposta da requisição *Fetch* em um objeto JavaScript utilizável.
- (4) Adiciona uma classe CSS a um nó do DOM, possibilitando mudança visual sem reescrever atributos de estilo diretamente.
- (1) Cria um novo nó de elemento HTML em memória, que ainda não está visível na tela.
- (2) Insere um nó de elemento no final da lista de filhos de um nó pai especificado, renderizando-o na tela.



Parte 4: Questão Prática / Discursiva

Questão 10: Leitura de Código com Fetch API

Analise o trecho de código JavaScript abaixo:

```
const divResultado = document.querySelector('#resultado');

fetch('https://api.exemplo.com/usuario/1')
  .then(resposta => resposta.json())
  .then(dados => {
    const paragrafo = document.createElement('p');
    paragrafo.innerText = `Nome: ${dados.nome}`;
    divResultado.appendChild(paragrafo);
  })
  .catch(erro => {
    console.log('Ocorreu um erro');
  });
```

Explique com suas palavras o que este código faz passo a passo. O que o usuário veria na tela (supondo que no HTML exista a div correspondente e a requisição seja um sucesso retornando um JSON com {"nome": "Maria"})?

— Resposta esperada: 1. O código seleciona a div de ID "resultado" do HTML. 2. Faz uma requisição assíncrona para a API usando fetch. 3. Quando a resposta chega, ela é convertida para JSON no primeiro then(). 4. No segundo then(), o código cria dinamicamente uma tag <p>, adiciona o texto "Nome: Maria" (usando o dado que veio da API) e insere esse parágrafo dentro da div "resultado" usando appendChild. 5. O usuário veria aparecer na tela o texto "Nome: Maria".